

1. Create Database

```
CREATE DATABASE training_sql_db;  
USE training_sql_db;
```

2. Table 1: Books

Use this for:

SELECT, WHERE, DISTINCT, IN, BETWEEN, LIKE, ORDER BY, aggregate functions.

```
CREATE TABLE books  
(  
    book_id INT PRIMARY KEY,  
    book_title VARCHAR(100),  
    category VARCHAR(50),  
    author VARCHAR(50),  
    price DECIMAL(10,2),  
    stock INT,  
    published_year INT  
);
```

```
INSERT INTO books VALUES  
(1, 'Python Basics', 'Programming', 'Ravi Kumar', 550, 30, 2021),  
(2, 'Advanced SQL', 'Database', 'Priya Sharma', 750, 15, 2020),  
(3, 'Data Engineering Guide', 'Data', 'Amit Verma', 1200, 10, 2023),  
(4, 'Machine Learning Start', 'AI', 'Neha Reddy', 950, 8, 2022),  
(5, 'Excel for Business', 'Business', 'Kiran Rao', 400, 50, 2019),  
(6, 'Power BI Reports', 'Data', 'Sneha Patel', 850, 12, 2021),  
(7, 'Java Fundamentals', 'Programming', 'Arjun Mehta', 600, 20, 2018),  
(8, 'Cloud Basics', 'Cloud', 'Rahul Nair', 700, 18, 2022),  
(9, 'SQL Interview Prep', 'Database', 'Farhan Ali', 500, 25, 2024),  
(10, 'AI for Beginners', 'AI', 'Meera Singh', 650, 5, 2023);
```

Exercises: SELECT, WHERE, DISTINCT, IN, BETWEEN, LIKE, ORDER BY

Exercise 1

Display all books.

Exercise 2

Display only:

```
book_title
category
price
```

Exercise 3

Display all unique book categories.

Exercise 4

Find all books from the `Programming` category.

Exercise 5

Find books priced above ₹700.

Exercise 6

Find books where stock is less than 15.

Exercise 7

Find books from these categories:

```
Programming
Database
AI
```

Use `IN` .

Exercise 8

Find books priced between ₹500 and ₹900.

Exercise 9

Find books whose title contains `SQL` .

Exercise 10

Find books whose title starts with `Data` .

Exercise 11

Sort books by price from highest to lowest.

Exercise 12

Sort books by category ascending and price descending.

Exercises: Aggregate Functions, GROUP BY, HAVING

Exercise 13

Find total number of books.

Exercise 14

Find highest book price.

Exercise 15

Find lowest book price.

Exercise 16

Find average book price.

Exercise 17

Find total stock available.

Exercise 18

Display number of books in each category.

Exercise 19

Display average price by category.

Exercise 20

Display total stock by category.

Exercise 21

Display categories having more than 1 book.

Exercise 22

Display categories where average price is greater than ₹700.

3. Table 2 and 3: Departments and Employees

Use this for JOINS and NULL handling.

```
CREATE TABLE departments
(
    department_id INT PRIMARY KEY,
    department_name VARCHAR(50),
    location VARCHAR(50)
);
```

```
CREATE TABLE employees
(
    employee_id INT PRIMARY KEY,
    employee_name VARCHAR(50),
    department_id INT,
    salary DECIMAL(10,2),
    city VARCHAR(50),
    manager_id INT
);
```

```
INSERT INTO departments VALUES
(10, 'IT', 'Hyderabad'),
(20, 'HR', 'Bangalore'),
(30, 'Finance', 'Mumbai'),
(40, 'Sales', 'Delhi'),
(50, 'Marketing', NULL);
```

```
INSERT INTO employees VALUES
(101, 'Rahul Sharma', 10, 75000, 'Hyderabad', 201),
(102, 'Priya Reddy', 10, 85000, 'Bangalore', 201),
(103, 'Amit Kumar', 20, 55000, NULL, 202),
(104, 'Sneha Patel', 30, 65000, 'Mumbai', 203),
(105, 'Arjun Verma', NULL, 60000, 'Chennai', 204),
(106, 'Neha Singh', 60, 50000, 'Delhi', NULL),
(107, 'Farhan Ali', 40, NULL, 'Hyderabad', 205),
(108, 'Meera Nair', 10, 90000, 'Pune', 201);
```

Exercises: INNER JOIN, LEFT JOIN, RIGHT JOIN

Exercise 23

Display employee name, salary, department name and department location using `INNER JOIN`.

Exercise 24

Display all employees and their department details using `LEFT JOIN`.

Exercise 25

Find employees who do not have a valid department.

Expected:

```
Arjun Verma  
Neha Singh
```

Exercise 26

Display all departments and employees using `RIGHT JOIN`.

Exercise 27

Find departments where no employees are assigned.

Expected:

```
Marketing
```

Exercise 28

Find employees where salary is NULL.

Exercise 29

Find employees where city is NULL.

Exercise 30

Find departments where location is NULL.

Exercise 31

Display department-wise employee count.

Exercise 32

Display department-wise average salary.

Exercise 33

Display departments having more than 2 employees.

Exercise 34

Display department-wise highest salary.

4. Table 4: Customers and Payments

Use this for subqueries.

```
CREATE TABLE customers_new  
(
```

```
customer_id INT PRIMARY KEY,  
customer_name VARCHAR(50),  
city VARCHAR(50),  
membership_type VARCHAR(30)  
);
```

```
CREATE TABLE payments  
(  
    payment_id INT PRIMARY KEY,  
    customer_id INT,  
    amount DECIMAL(10,2),  
    payment_mode VARCHAR(30),  
    payment_status VARCHAR(30)  
);
```

```
INSERT INTO customers_new VALUES  
(1, 'Ramesh Gupta', 'Hyderabad', 'Gold'),  
(2, 'Sana Khan', 'Bangalore', 'Silver'),  
(3, 'John Mathew', 'Mumbai', 'Gold'),  
(4, 'Ayesha Begum', 'Chennai', 'Bronze'),  
(5, 'Vikram Rao', 'Delhi', 'Silver'),  
(6, 'Divya Sharma', 'Pune', NULL);
```

```
INSERT INTO payments VALUES  
(1001, 1, 15000, 'UPI', 'Success'),  
(1002, 1, 8000, 'Card', 'Success'),  
(1003, 2, 5000, 'Cash', 'Pending'),  
(1004, 3, 22000, 'UPI', 'Success'),  
(1005, 7, 12000, 'Card', 'Failed'),  
(1006, NULL, 3000, 'Cash', 'Pending'),  
(1007, 4, NULL, 'UPI', 'Success'),  
(1008, 5, 7000, NULL, 'Success');
```

Exercises: Subqueries

Exercise 35

Find customers who made payments.

Use `IN` .

Exercise 36

Find customers who never made payments.

Use `NOT IN` or `NOT EXISTS` .

Exercise 37

Find payments greater than average payment amount.

Exercise 38

Find customer who made the highest payment.

Exercise 39

Find Gold customers who made payments.

Exercise 40

Find customers whose total payment is greater than ₹10,000.

Exercise 41

Find payments where customer does not exist.

Expected:

```
payment_id 1005  
payment_id 1006
```

Exercise 42

Find customers using `EXISTS` .

Exercise 43

Find customers using `NOT EXISTS` .

Exercise 44

Find customers whose payment amount is greater than all payments made by customer 2.
